

Scaling Techniques Applied to the Customer Churn Dataset

WILDA Team 4 · ACS Work Integrated Learning, Data Analytics · Stage 2 Data Preparation · Rohan Sharma Kharel

1. Purpose: why scaling matters here

After encoding, the dataset has 10 features and one target. Eight of those features are 0/1 flags, but two are not: **tenure** runs from 0 to 72 months and **MonthlyCharges** runs from \$18 to \$119. Left as they are, those two columns are up to two orders of magnitude larger than every flag beside them. That matters because of what happens next. The clustering stage uses K-Means, which assigns a customer to a cluster by straight-line distance, so a column measured in dollars would dominate the distance calculation and the clusters would be little more than a split on price. The Stage 3 neural network is affected too: inputs on widely different scales make the gradients pull unevenly and the network slower and less stable to train. Putting the continuous columns on the same footing as the flags is what stops the unit of measurement deciding the result.

2. Technique chosen: StandardScaler (z-score standardisation)

Each value is replaced by how many standard deviations it sits from its column mean: $z = (x - \text{mean}) / \text{std}$. The transformed column has a mean of 0 and a standard deviation of 1. Two alternatives were considered and rejected:

- **MinMaxScaler** squeezes a column into the 0 to 1 range using only its minimum and maximum, so the two most extreme customers set the scale for everyone else and the rest of the distribution is compressed. StandardScaler uses every row, keeps the shape of the distribution intact, and produces the zero-centred inputs that suit the Stage 3 neural network.
- **RobustScaler** uses the median and the interquartile range and earns its keep when extreme outliers would distort the mean and standard deviation. Neither column has that problem: tenure is capped at 72 months by the data itself and MonthlyCharges tops out at \$119, so there is nothing for a robust method to protect against.

3. Columns scaled and not scaled

Only **tenure** and **MonthlyCharges** are scaled. The eight encoded columns (gender, SeniorCitizen, Dependents, PhoneService, MultipleLines, InternetService, Contract_One_year, Contract_Two_year) are already 0/1, so they are on a comparable range with each other and with the standardised columns. Standardising a binary flag adds nothing: it just replaces the two values with two other numbers, while making the column harder to read and harder to explain to the client. The target **Churn** is never scaled, because it is the label being predicted, not an input feature.

4. Preventing data leakage

The scaler is fitted on the training set only, and that same fitted scaler is then used to transform the test set. The order matters: the data is split first and scaled second. Fitting the scaler on all 7,043 rows would mean the mean and standard deviation it subtracts and divides by were calculated partly from the test rows. The test set is meant to stand in for customers the model has never seen, so the moment its statistics help shape the training data, the evaluation is measuring something easier than reality and the reported accuracy is optimistic. Fitting on training data only is also what actually happens in production: when a new customer arrives tomorrow, the only statistics available are the ones already learned.

5. The split the scaler was fitted on

The dataset was split 80/20, stratified on Churn, with `random_state=42` for reproducibility. Churn is imbalanced at 26.54%, so stratifying is what keeps that rate steady across both sets:

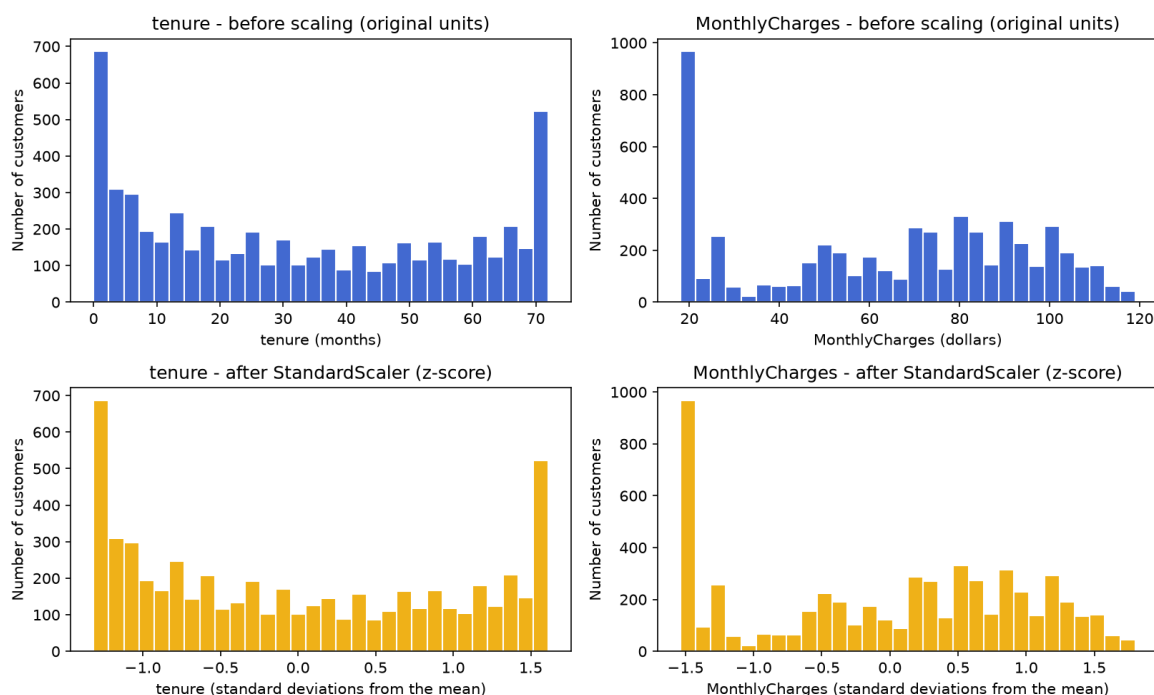
Set	Rows	% of total	Churners	Churn rate
Full dataset	7,043	100.00%	1,869	26.54%
Training set	5,634	79.99%	1,495	26.54%
Test set	1,409	20.01%	374	26.54%

6. Results

Column	Train mean before	Train std before	Train mean after	Train std after	Test mean after	Test std after
tenure (months)	32.3603	24.5639	0.0000	1.0000	0.0022	0.9987
MonthlyCharges (\$)	64.5351	30.2583	0.0000	1.0000	0.0369	0.9712

The training columns come out at exactly mean 0 and standard deviation 1, which is what fitting on that set guarantees. The test set lands close to but not exactly on 0 and 1 (tenure 0.0022 and 0.9987, MonthlyCharges 0.0369 and 0.9712). That is the expected result and a useful check: if the test columns were also exactly 0 and 1, the scaler would have been fitted on the full dataset and the pipeline would be leaking. The figure below shows the effect on the training set. The bars keep their shape exactly; only the axis changes, from months and dollars to standard deviations from the mean.

Training set before and after standardisation



Training set distributions before (top) and after (bottom) standardisation.

7. Code snippets

Taken from `Data_Preparation/data_preparation.py`. First the split, which happens before any scaling:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
)
```

Then the scaler, fitted on the training rows and applied to both sets. `fit_transform` is called on the training data only; the test data gets `transform`, which reuses the statistics already learned:

```
scaler = StandardScaler()
X_train[SCALE_COLS] = scaler.fit_transform(X_train[SCALE_COLS]) # fit on TRAIN
X_test[SCALE_COLS] = scaler.transform(X_test[SCALE_COLS])        # apply to TEST
```

Finally the fitted scaler is saved so it can be reused:

```
joblib.dump(scaler, OUT_DIR / "scaler.pkl")
```

8. Reusing the scaler downstream

`scaler.pkl` holds the means and standard deviations learned from the training set (tenure 32.3603 / 24.5639, MonthlyCharges 64.5351 / 30.2583). The clustering stage and the Stage 3 model load it rather than fitting their own, so every stage transforms data identically and a cluster centre can be converted back into months and dollars for the client:

```
import joblib

scaler = joblib.load("Data_Preparation/scaler.pkl")
SCALE_COLS = ["tenure", "MonthlyCharges"]

# Apply the identical transformation to any new data
new_data[SCALE_COLS] = scaler.transform(new_data[SCALE_COLS])

# Or read a scaled value back in months and dollars
original = scaler.inverse_transform(scaled_values)
```

The pre-scaled train and test sets are committed as `train_set.csv` and `test_set.csv`, so the later stages do not need to repeat any of this; the scaler is there for new data and for reading results back in the original units. The row counts, churn rates and scaler statistics quoted throughout this document are generated from `preparation_summary.json`, which the pipeline writes on each run.